



Python Programming Fundamentals

List Methods

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. Adding Items
2. Removing Items
3. Sorting and Reversing
4. Searching
5. Copying Lists
6. All List Methods Summary
7. Practical Example — To-Do List



Outline

1. Adding Items

2. Removing Items

3. Sorting and Reversing

4. Searching

5. Copying Lists

6. All List Methods Summary

7. Practical Example — To-Do List



1. Adding Items

```
numbers = [1, 2, 3]
```

```
# append(item) – add one item to the END
```

```
numbers.append(4)
```

```
print(numbers)    # [1, 2, 3, 4]
```

```
# extend(iterable) – add all items from another iterable
```

```
numbers.extend([5, 6, 7])
```

```
print(numbers)    # [1, 2, 3, 4, 5, 6, 7]
```

```
# insert(index, item) – insert at a specific position
```

```
numbers.insert(0, 0)    # insert 0 at position 0
```

```
print(numbers)    # [0, 1, 2, 3, 4, 5, 6, 7]
```

```
numbers.insert(4, 99)    # insert 99 at position 4
```

```
print(numbers)    # [0, 1, 2, 3, 99, 4, 5, 6, 7]
```



Outline

1. Adding Items
- 2. Removing Items**
3. Sorting and Reversing
4. Searching
5. Copying Lists
6. All List Methods Summary
7. Practical Example — To-Do List



2. Removing Items

```
fruits = ["apple", "banana", "cherry", "banana", "date"]
```

```
# remove(value) – removes FIRST occurrence of value
```

```
fruits.remove("banana")
```

```
print(fruits)    # ['apple', 'cherry', 'banana', 'date']
```

```
# pop(index) – removes AND returns item at index
```

```
last = fruits.pop()    # no index = last item
```

```
print(last)    # date
```

```
print(fruits)    # ['apple', 'cherry', 'banana']
```

```
second = fruits.pop(1)    # remove at index 1
```

```
print(second)    # cherry
```

```
# clear() – removes ALL items
```

```
numbers = [1, 2, 3, 4, 5]
```



2. Removing Items

```
numbers.clear()  
print(numbers)  # []
```



Outline

1. Adding Items
2. Removing Items
- 3. Sorting and Reversing**
4. Searching
5. Copying Lists
6. All List Methods Summary
7. Practical Example — To-Do List



3. Sorting and Reversing

```
numbers = [5, 2, 8, 1, 9, 3]
```

```
# sort() – sorts IN PLACE (modifies the list)
```

```
numbers.sort()
```

```
print(numbers)           # [1, 2, 3, 5, 8, 9]
```

```
numbers.sort(reverse=True) # descending
```

```
print(numbers)           # [9, 8, 5, 3, 2, 1]
```

```
# reverse() – reverses IN PLACE
```

```
numbers.reverse()
```

```
print(numbers)           # [1, 2, 3, 5, 8, 9]
```



3. Sorting and Reversing



3. Sorting and Reversing

```
numbers = [5, 2, 8, 1, 9, 3]
```

```
# sort() – sorts IN PLACE (modifies the list)
```

```
numbers.sort()
```

```
print(numbers)           # [1, 2, 3, 5, 8, 9]
```

```
numbers.sort(reverse=True) # descending
```

```
print(numbers)           # [9, 8, 5, 3, 2, 1]
```

```
# reverse() – reverses IN PLACE
```

```
numbers.reverse()
```

```
print(numbers)           # [1, 2, 3, 5, 8, 9]
```

```
# sorted() – returns a NEW sorted list (original unchanged)
```

```
words = ["banana", "apple", "cherry", "date"]
```

```
sorted_words = sorted(words)
```



3. Sorting and Reversing

```
print(words)          # ['banana', 'apple', 'cherry', 'date']
print(sorted_words)   # ['apple', 'banana', 'cherry', 'date']

# Sort by string length
by_length = sorted(words, key=len)
print(by_length)      # ['date', 'apple', 'banana', 'cherry']
```



Outline

1. Adding Items
2. Removing Items
3. Sorting and Reversing
- 4. Searching**
5. Copying Lists
6. All List Methods Summary
7. Practical Example — To-Do List



4. Searching

```
fruits = ["apple", "banana", "cherry", "banana", "apple"]
```

```
# count(value) – how many times value appears
```

```
print(fruits.count("banana")) # 2
```

```
print(fruits.count("grape")) # 0
```

```
# index(value) – position of FIRST occurrence
```

```
print(fruits.index("cherry")) # 2
```

```
print(fruits.index("banana")) # 1 (first banana)
```

```
# Searching safely
```

```
def find_item(lst, value):
```

```
    if value in lst:
```

```
        return lst.index(value)
```

```
    return -1
```



4. Searching

```
print(find_item(fruits, "apple"))    # 0  
print(find_item(fruits, "grape"))    # -1
```



Outline

1. Adding Items
2. Removing Items
3. Sorting and Reversing
4. Searching
- 5. Copying Lists**
6. All List Methods Summary
7. Practical Example — To-Do List



5. Copying Lists

```
original = [1, 2, 3, 4, 5]
```

```
# WRONG – this creates an alias, not a copy!
```

```
alias = original
```

```
alias.append(99)
```

```
print(original) # [1, 2, 3, 4, 5, 99] – original changed!
```

```
# Correct ways to copy
```

```
copy1 = original.copy() # method
```

```
copy2 = original[:] # slice
```

```
copy3 = list(original) # constructor
```

```
copy1.append(100)
```

```
print(original) # unchanged
```

```
print(copy1) # [1, 2, 3, 4, 5, 99, 100]
```



Outline

1. Adding Items
2. Removing Items
3. Sorting and Reversing
4. Searching
5. Copying Lists
- 6. All List Methods Summary**
7. Practical Example — To-Do List



6. All List Methods Summary

Method	Description
<code>append(x)</code>	Add x to end
<code>extend(it)</code>	Add all items from iterable
<code>insert(i, x)</code>	Insert x at position i
<code>remove(x)</code>	Remove first occurrence of x
<code>pop(i)</code>	Remove and return item at i (default: last)
<code>clear()</code>	Remove all items
<code>sort()</code>	Sort in place
<code>reverse()</code>	Reverse in place
<code>index(x)</code>	Return index of first x
<code>count(x)</code>	Count occurrences of x
<code>copy()</code>	Return a shallow copy



Outline

1. Adding Items
2. Removing Items
3. Sorting and Reversing
4. Searching
5. Copying Lists
6. All List Methods Summary
- 7. Practical Example — To-Do List**



7. Practical Example — To-Do List

```
def show_todos(todos):  
    if not todos:  
        print("No tasks!")  
        return  
    for i, task in enumerate(todos, start=1):  
        print(f" {i}. {task}")  
  
todos = []  
  
while True:  
    print("\n1. Add task  2. Remove task  3. Show tasks  4. Quit")  
    choice = input("Choice: ")  
  
    if choice == "1":  
        task = input("Task: ").strip()  
        todos.append(task)
```



7. Practical Example — To-Do List

```
elif choice == "2":
    show_todos(todos)
    idx = int(input("Remove task number: ")) - 1
    if 0 <= idx < len(todos):
        removed = todos.pop(idx)
        print(f"Removed: {removed}")
elif choice == "3":
    show_todos(todos)
elif choice == "4":
    break
```

Thanks for Watching - Any questions?

